

Further aspects of calculation of instantaneous frequency from the PM data or from the vibration signal itself are presented in Section 4.4.

4.2.3 Integration

Numerical integration is a very broad subject, and it is well-described in the literature [4]. However, when CMS-data is considered, numerical integration is usually used to convert **Acceleration** vibration signal to **Velocity** signal, and sometimes to convert **Velocity** vibration signal to **Displacement** signal, which limits the scope of integration methods. These complete processes are described in Section 4.3.2 with full-featured source codes.

The current Section illustrates three selected techniques for signal integration, namely:

- `cumsum` function,
- `cumtrapz` function
- user quick function:

```
function [xInt] = Calculate_Integrated(x, fs)

% COMMENTS
% Earlier HP signal filtering recommended.
% Taking "dt" out of the loop changes result in MATLAB of the (e-10) order.

% INPUT
% x - real time signal <1xN>
% fs - sampling frequency <1x1>

% OUTPUT
% x - integrated real time signal <1xN>

x = x(:);           % force column vector
N = length(x);      % length of the original signal
x = x - mean(x);     % remove DC component before integration

xInt = x;           % cumulative variable
xInt(1) = 0;         % set first point manually to "0"

for i = 2:N
    xInt(i) = xInt(i-1)+x(i)/fs;    % division by fs is multiplication by dt
end

xInt = xInt - mean(xInt);           % remove DC component after integration
```

Worth mentioning, practical implementation of integration within machine diagnostics typically involves a priori signal filtering using a high-pass filter to avoid cumulating of numerically-generated signal distortion, which could be understood as a low-order or low-frequency artifacts.

4.2.3.1 Noise-free synthetic signal

Figure 4.29 illustrates a noise-free sinusoidal component and its integrals calculated using three considered integration techniques. The top plot in Figure 4.29 is generated when the mean value of integrated signal is not removed after integration using the `cumsum` and `cumtrapz` functions. The bottom plot in Figure 4.29 represents integrated signals, where mean (DC) component is removed after integration.

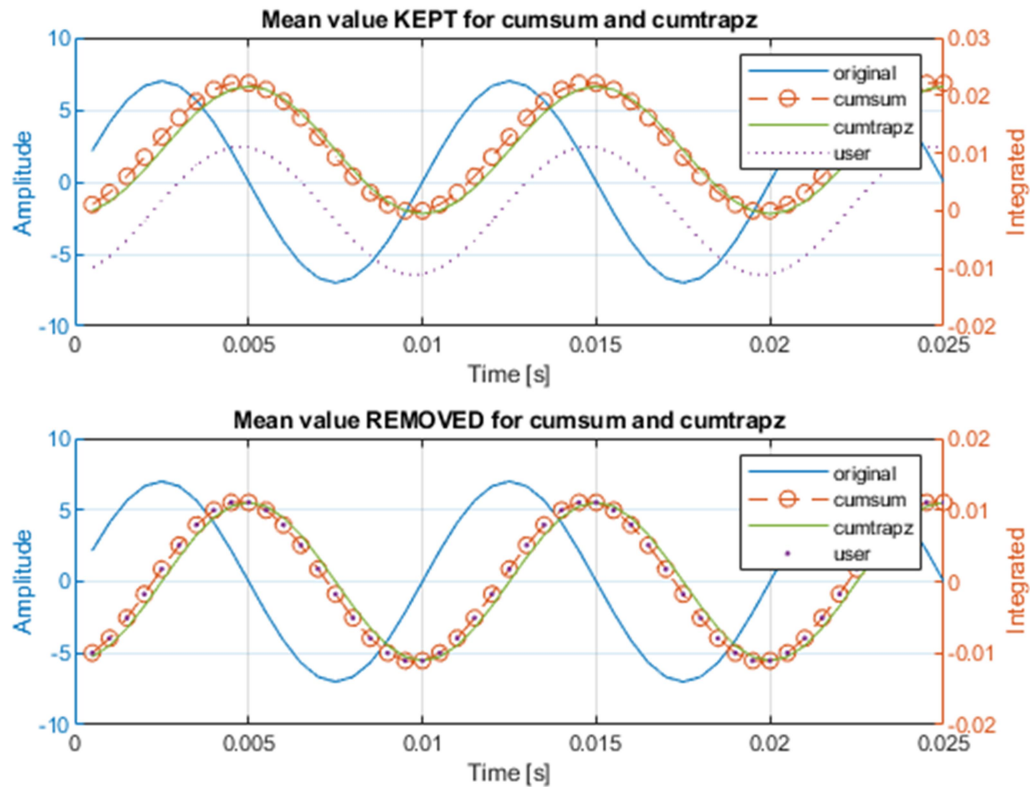


Figure 4.29 Comparison of integration techniques for noise-free signal (USER zoom)

The difference between in results between the `cumtrapz` function and the `cumsum` function are at the order of 10^{-3} :

```
>> figure; plot(Int_cs - Int_ct)
```

However, this difference is due to the phase shift. When amplitudes are considered:

```
>> max(Int_cs) - max(Int_ct),
```

the difference is at the order of 10^{-15} . The difference in results between the `cumsum` function and `user Calculate_Integrated` function (for this example) are at the order of 10^{-15} :

```
>> figure; plot(Int_cs - Int_user)
```

Therefore, it might be accepted that from the diagnostic point-of-view, these differences are negligible, and any standard integration algorithm is generally suitable. Figure 4.29 is generated with following code:

```
% Define colors
MatlabColors = colororder;
c1 = MatlabColors(1,:);
c2 = MatlabColors(2,:);
c3 = MatlabColors(5,:);
c4 = MatlabColors(4,:);

% Necessary when yyaxis is used
% USER color #1
% USER color #2
% USER color #3
% USER color #4
```

```

% Signal parameters
fs = 2000; % USER sampling frequency
dt = 1/fs; % time resolution
T = 10; % USER total signal time
t = dt:dt:T; % time base

% ----- fragment of the code to be modified -----
x = 7*sin(2*pi*100*t); % USER noise-free signal
% -----

Int_cs = cumsum(x) /fs; % integrate using cumulative sum
Int_ct = cumtrapz(x)/fs; % integrate using "cumulative
% trapezoidal numerical
integration"
Int_user = Calculate_Integrated(x,fs)'; % integrate using USER function

% PLOT DATA where MEAN value is KEPT for CUMSUM and CUMTRAPZ
figure;subplot(211);
yyaxis left; plot(t,x,'color',c1); grid on;
xlim([0 .025]);
xlabel('Time [s]');
ylabel('Amplitude');
title('Mean value KEPT for cumsum and cumtrapz')

yyaxis right; plot(t,Int_cs, '--o','color',c2); hold on;
plot(t,Int_ct, '-', 'color',c3);
plot(t,Int_user, '.', 'color',c4);
legend('original','cumsum','cumtrapz','user');
ylabel('Integrated');
set(gca,'fontsize',8);

% PLOT DATA where MEAN value is REMOVED for CUMSUM and CUMTRAPZ
Int_cs = Int_cs - mean(Int_cs); % remove mean value
Int_ct = Int_ct - mean(Int_ct); % remove mean value

subplot(212);
yyaxis left; plot(t,x,'color',c1); grid on;
xlim([0 .025]);
xlabel('Time [s]');
ylabel('Amplitude');
title('Mean value REMOVED for cumsum and cumtrapz')

yyaxis right; plot(t,Int_cs, '--o','color',c2); hold on;
plot(t,Int_ct, '-', 'color',c3);
plot(t,Int_user, '.', 'color',c4);
legend('original','cumsum','cumtrapz','user');
ylabel('Integrated');
set(gca,'fontsize',8);

```

4.2.3.2 Noisy synthetic signal

When a random noise is added to the signal generated by a code from previous Section, i.e. when:

```

% ----- fragment of the code to be modified -----
x = 7*sin(2*pi*100*t); % USER noise-free signal
% -----

```

is replaced with:

```

% ----- fragment of the code to be modified -----
rng(5); % Use this rng value

```

```
x = 7*sin(2*pi*100*t) + 2*(rand(1,length(t))-0.5); % USER noisy signal
% -----
```

resultant plots for data with and without mean value removed after integration (NOTE: in full time scale) are as presented in Figure 4.30.

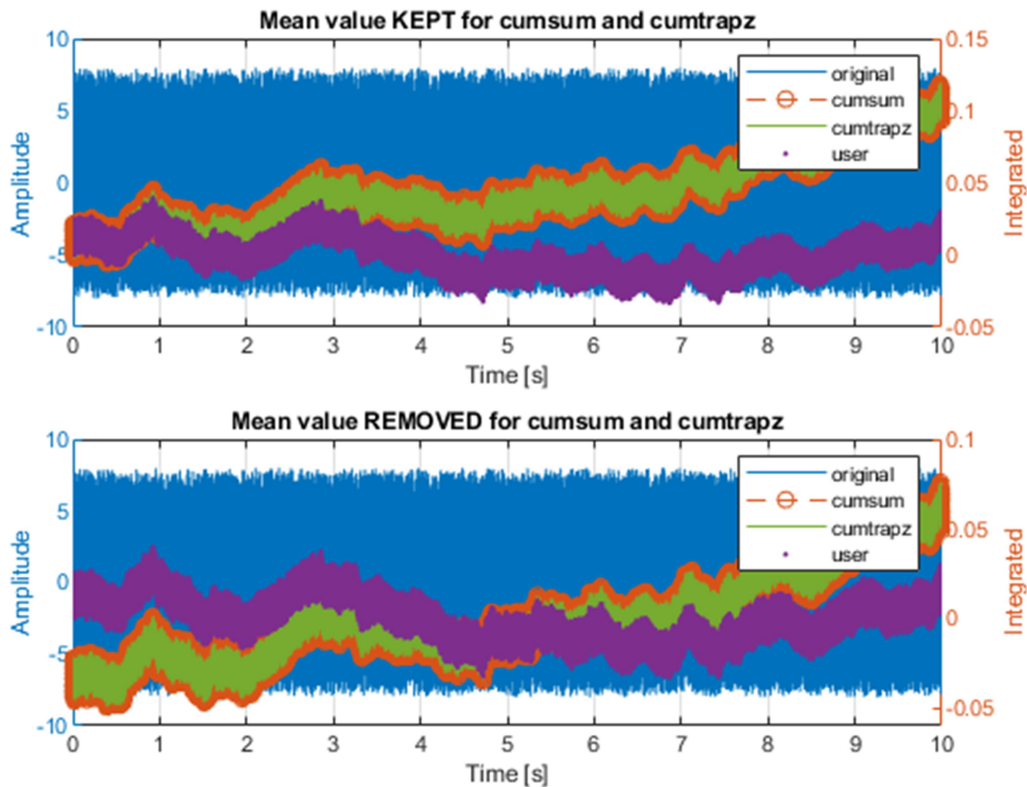


Figure 4.30 Comparison of integration techniques for noisy signal

Clearly, the user function performs better, because the integrated signal should oscillate around zero the same way as the original signal does. However, this improvement is due only to following line, which is inside the user `Calculate_Integrated` function:

```
x = x - mean(x); % remove DC component before integration
```

This problem is generally solved by initial HP signal filtering¹⁰, for instance:

```
% ----- fragment of the code to be modified -----
rng(5); % Use this rng value
x = 7*sin(2*pi*100*t) + 2*(rand(1,length(t))-0.5); % USER noisy signal
x = highpass(x,10,fs); % filter the signal with 10 Hz cutoff frequency
% -----
```

¹⁰ NOTE: for conciseness, in this Section, HP filtering is done via `highpass` function. In other places, function `butter` with manual parameters or `Filter_signal_HP` user function is used

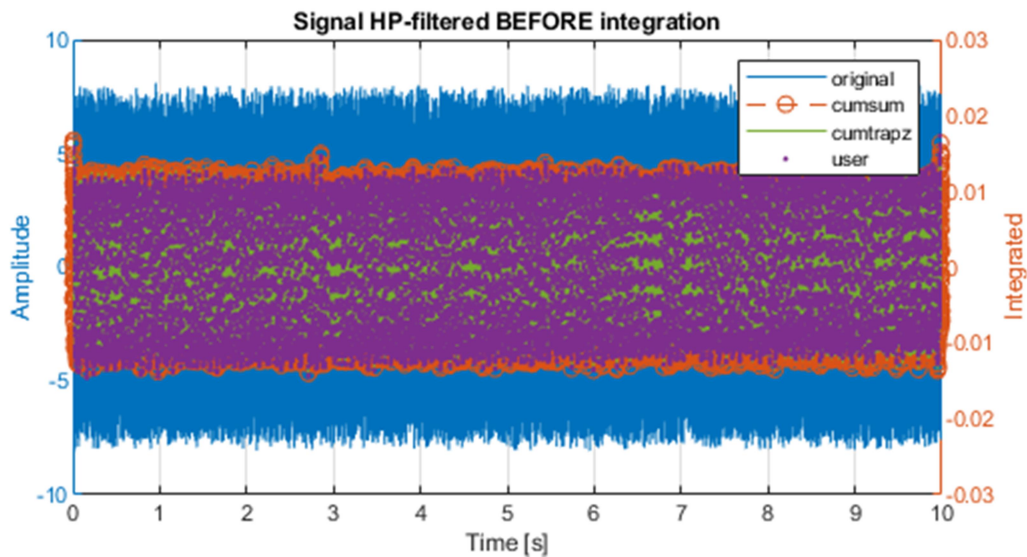


Figure 4.31 Comparison of integration techniques for noisy HP filtered signal

As observed in Figure 4.31, (right vertical axis is manually adjusted by calling `ylim([-0.03, 0.03])`), results of all considered integration techniques are visually the same. When detailed spectral analysis is considered, MATLAB® `cumsum` and `cumtrapz` functions show slightly better performance than the simple user `Calculate_Integrated` function, especially at frequencies close to zero.

4.2.3.3 Real signal

The current example illustrates practical integration of a real vibration signal collected from a REB housing with a general purpose industrial accelerometer¹¹. The machine experiences a slight speed variation, which is typical even for machinery working in “stationary” conditions. The data is collected using **25 kHz** sampling frequency. Figure 4.32 illustrates time waveforms of the original acceleration signal and its integrals using considered techniques. Different HP cutoff frequencies are analyzed for each technique. Note: HP filtering and integration are used when a velocity signal or displacement signal is calculated from the acceleration signal. Both processes are illustrated in details in Section 4.3.2.

¹¹ Available @ <http://home.agh.edu.pl/~ajab/>

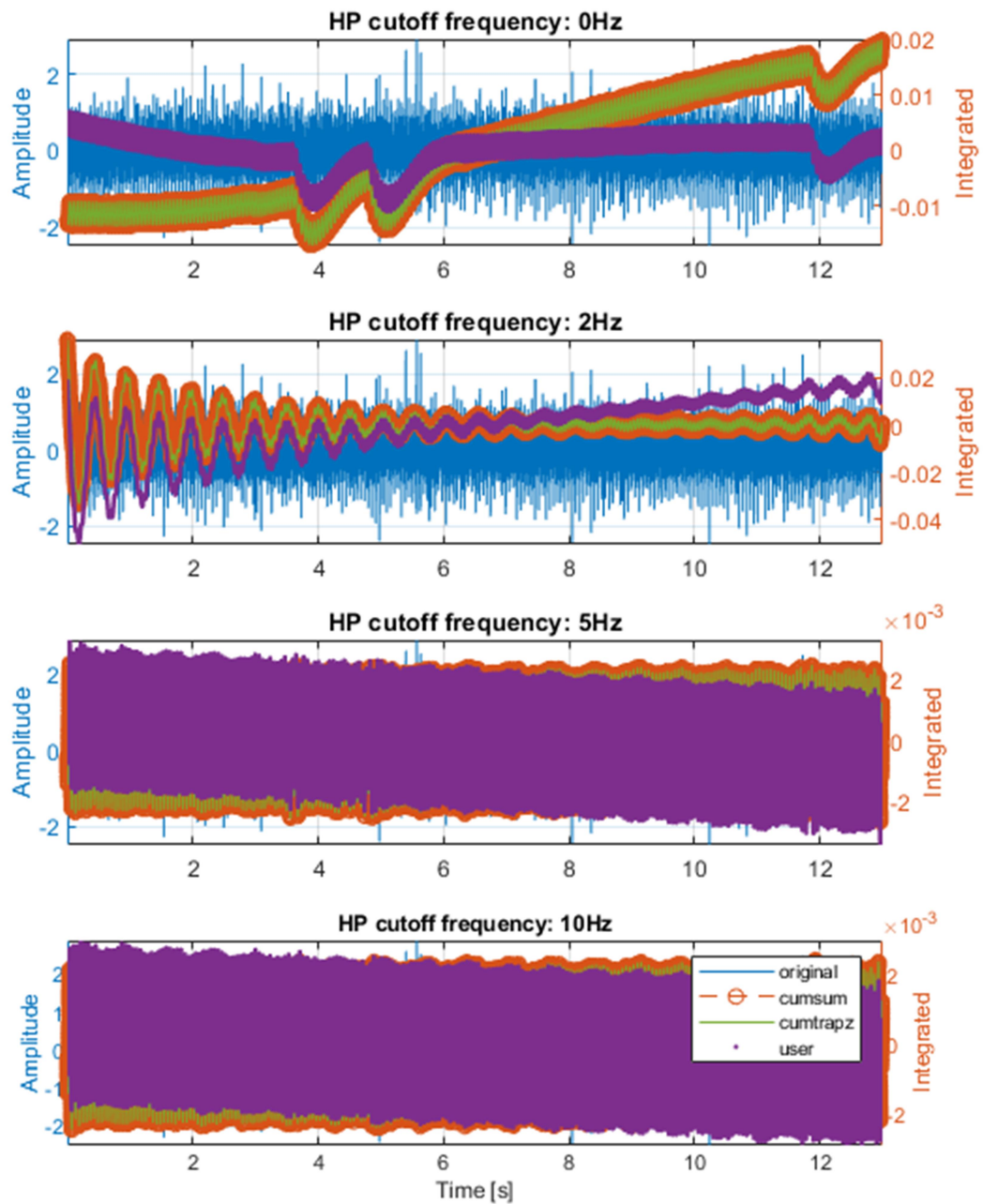


Figure 4.32 Analysis of different integration techniques with different HP cutoff frequency filter settings for real vibration signal analysis

As observed in Figure 4.32, real signals require relatively high HP cutoff frequencies, like **5 Hz**, regardless of the integration technique used. Note that the top two graphs have over order of magnitude larger amplitude range. Regarding CMS-oriented purposes, a HP filter cutoff frequency more than **10 Hz** is typically not used for integration, as velocity data is significant for relatively low frequencies (no more than 1 kHz), which is evidently a contradiction. As a general rule, the longer the signal to be integrated, the higher HP filter cutoff frequency is required. Finally, plots corresponding to **5 Hz** and **10 Hz** cutoff frequencies clearly show that integration using MATLAB® `cumsum` or

cumtrapz functions is practically free from low-frequency artifacts. However, since these particular functions are generally not available outside MATLAB®, the user function Calculate_Integrated is used throughout this book for referential purposes.

Figure 4.32 is generated using following code:

```
load vibstand.mat; % Available @ http://home.agh.edu.pl/~ajab/
% Define colors
MatlabColors = colororder; % Necessary when yyaxis is used
c1 = MatlabColors(1,:); % USER color #1
c2 = MatlabColors(2,:); % USER color #2
c3 = MatlabColors(5,:); % USER color #3
c4 = MatlabColors(4,:); % USER color #4

x = avtb_2019_Machine_1_02_Motor_bearing(:,2); % time waveform
fs = 25e3; % sampling frequency
dt = 1/fs; % time resolution
N = length(x); % No. of samples
T = N*dt; % total signal time [s]
t = dt:dt:T; % time base

% USER filter cutoff frequencies
HP_cutoff(1) = 0; % no HP filtering
HP_cutoff(2) = 2; % first common HP filter cutoff frequency: 2Hz
HP_cutoff(3) = 5; % second common HP filter cutoff frequency: 5Hz
HP_cutoff(4) = 10; % third common HP filter cutoff frequency: 10Hz

% FILTER SIGNAL
y(:,1) = x; % original signal
y(:,2) = highpass(x,HP_cutoff(2),fs); % filter the signal (quick way)
y(:,3) = highpass(x,HP_cutoff(3),fs); % filter the signal (quick way)
y(:,4) = highpass(x,HP_cutoff(4),fs); % filter the signal (quick way)

% INTEGRATE using different methods and PLOT
for k = 1:4
    Int_cs = cumsum(y(:,k)) /fs; Int_cs = Int_cs - mean(Int_cs);
    Int_ct = cumtrapz(y(:,k))/fs; Int_ct = Int_ct - mean(Int_ct);
    Int_user = Calculate_Integrated(y(:,k),fs)';

    subplot(4,1,k);
    yyaxis left; plot(t,x,'color',c1); grid on;
    ylabel('Amplitude');
    str = ['HP cutoff frequency: ',num2str(HP_cutoff(k)),'Hz'];
    title(str);

    yyaxis right; plot(t,Int_cs, '--o','color',c2); hold on; axis tight;
    plot(t,Int_ct, '-', 'color',c3);
    plot(t,Int_user, '.', 'color',c4); ylabel('Integrated');
end
% FINAL LABELING
legend('original','cumsum','cumtrapz','user'); xlabel('Time [s]');
set(gca,'fontsize',8);
```

It is therefore concluded that for practical CMS-oriented implementation of signal integration, the HP filter settings and filter characteristics have generally more influence on the resultant integrated signal than the selected integration technique.